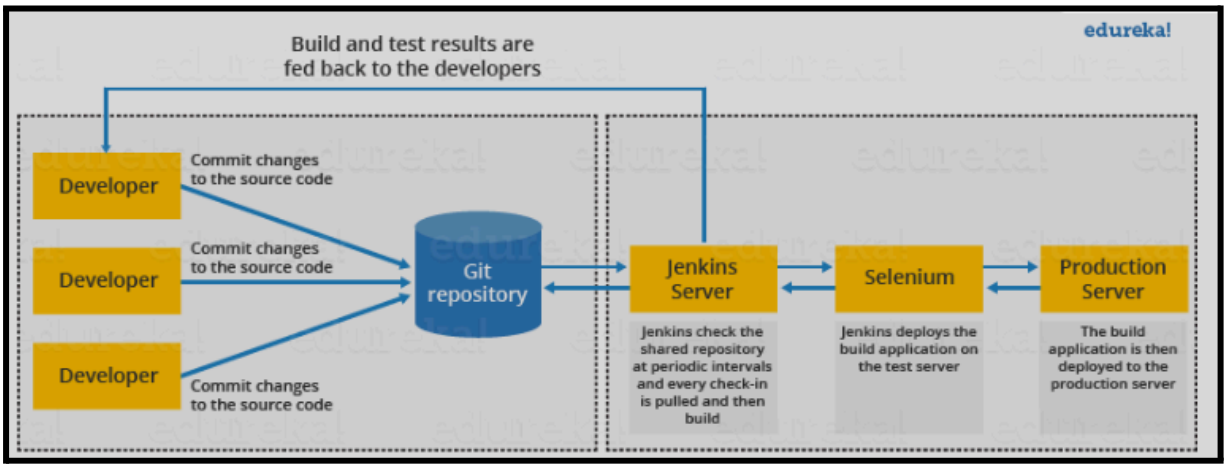
a. Answer the following:

1. Explain the architecture of Jenkins with a diagram.



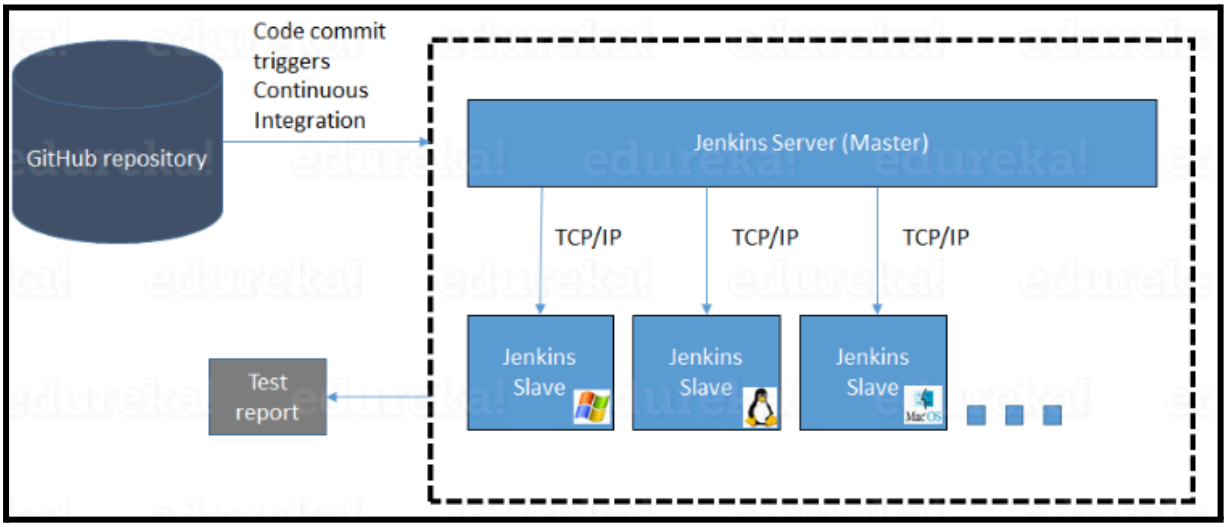
- At the beginning of the process, several developers write and modify the application source code. After completing their changes, they commit the updated code to a Git repository, which acts as a centralized version control system. The repository maintains different versions of the code and enables collaboration among developers by storing all commits in one place.

- The Jenkins server continuously monitors the Git repository for any changes. Whenever a developer commits new code, Jenkins detects the update and automatically pulls the latest version of the source code. Jenkins then starts the build process, where the code is compiled and packaged into a deployable artifact. This ensures that all newly committed code is integrated and verified through an automated build process.

- After a successful build, Jenkins deploys the application to a testing environment where Selenium is used for automated testing. Selenium executes predefined test cases to verify the functionality and behavior of the application. If the application passes all the tests, the validated build is then deployed to the production server, making it available for end users.

- Finally, the build and test results are fed back to the developers. This feedback mechanism helps developers quickly identify issues and fix them if the build or tests fail. Overall, this Jenkins architecture automates the process of integrating code, testing it, and deploying it, which improves development efficiency and ensures reliable software delivery.

2. Explain the distributed architecture of Jenkins with diagram



- In this architecture, developers commit their code to a GitHub repository, which acts as the centralized version control system. Each code commit triggers the Continuous Integration process. The Jenkins system detects these changes and initiates the build pipeline automatically.

- At the core of the system is the Jenkins Server (Master). The master node is responsible for managing the overall CI/CD process. It schedules jobs, monitors builds, distributes tasks, and maintains communication with other machines in the Jenkins environment. However, the master typically does not execute the heavy build tasks itself.

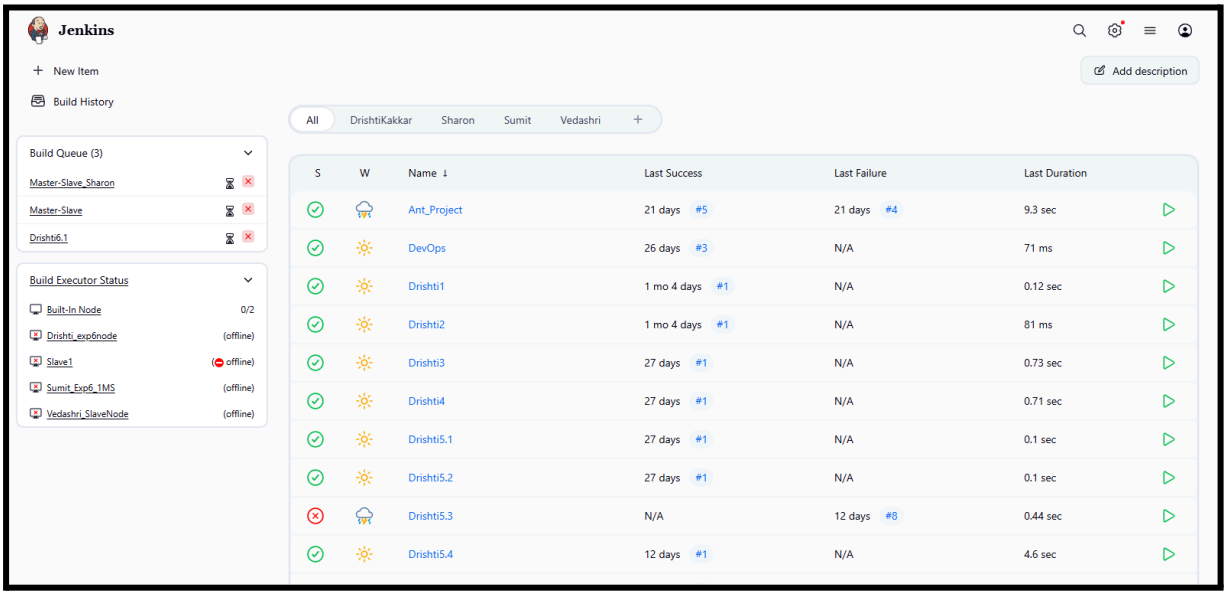
- The actual build and testing tasks are executed by Jenkins Slave nodes (also called agents). These slave machines connect to the master through TCP/IP communication. Each slave can run on different operating systems, such as Windows, Linux, or macOS, allowing Jenkins to test applications across multiple platforms. The master assigns build jobs to available slave nodes, enabling multiple builds or tests to run simultaneously.

- After the build and testing processes are completed on the slave nodes, the results and test reports are sent back to the Jenkins master. The master then records the outcomes and provides feedback to the development team.

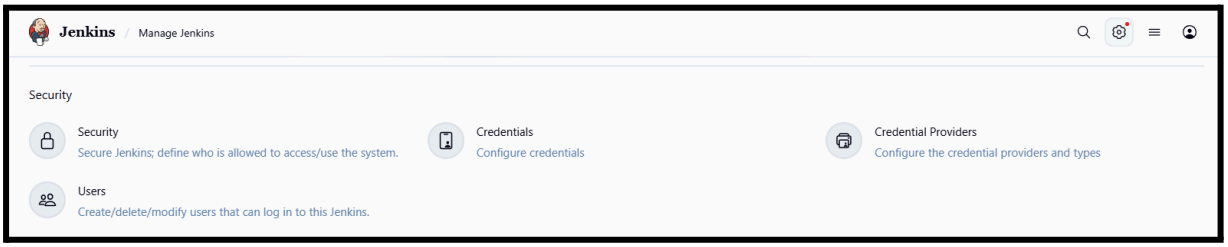
- Thus, the distributed architecture of Jenkins improves performance, scalability, and resource utilization by distributing build tasks across multiple machines while maintaining centralized control through the Jenkins master server.

b. Execute following (Refer the shared material) and attach screenshots:

Login to Jenkins



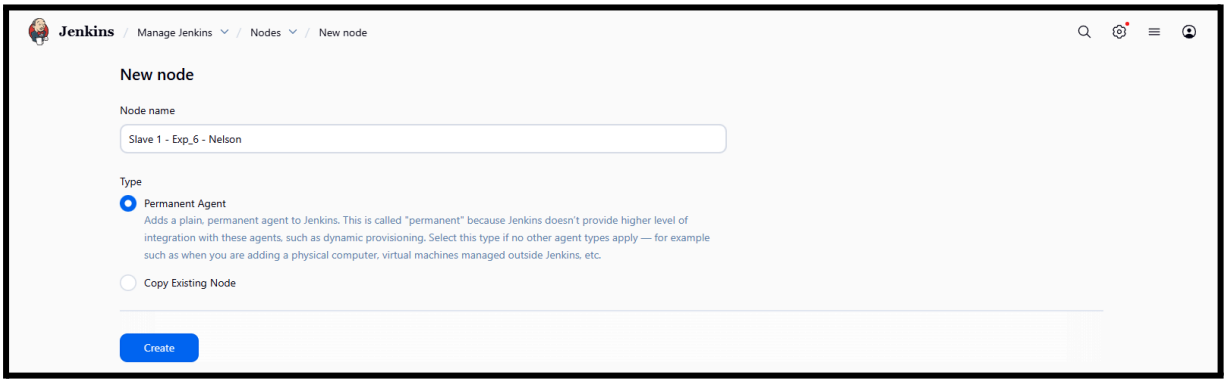
Go to Manage Jenkins —> Go to Security



In security, scroll till agents, click on Random for TCP port, apply and save

Jenkins -> manage Jenkins-> System Configuration ->Nodes
Go to new node

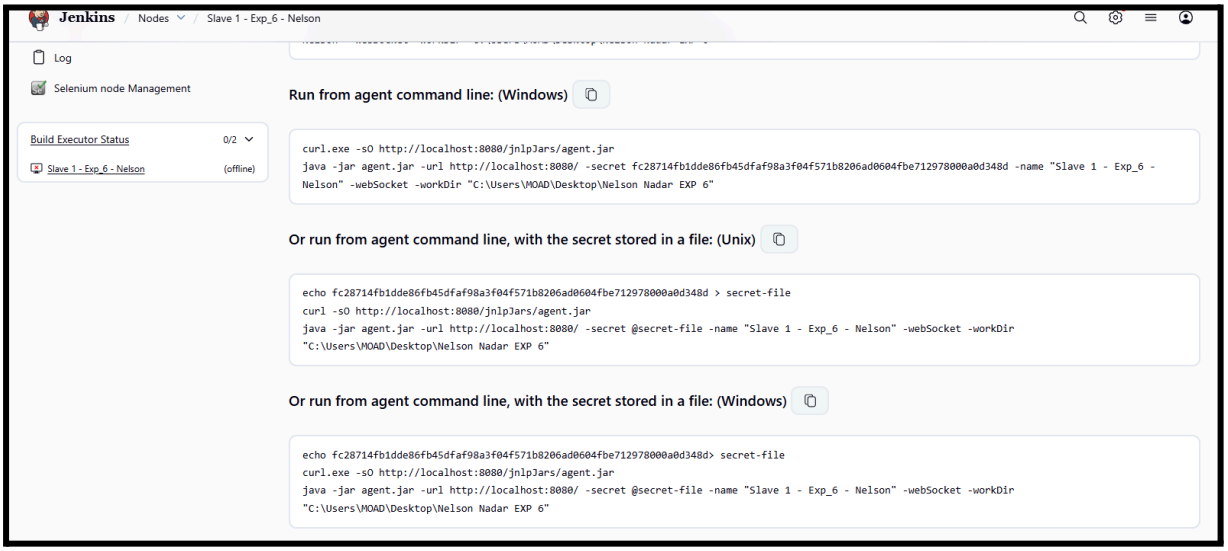
Give node name and click on permanent agent and create slave



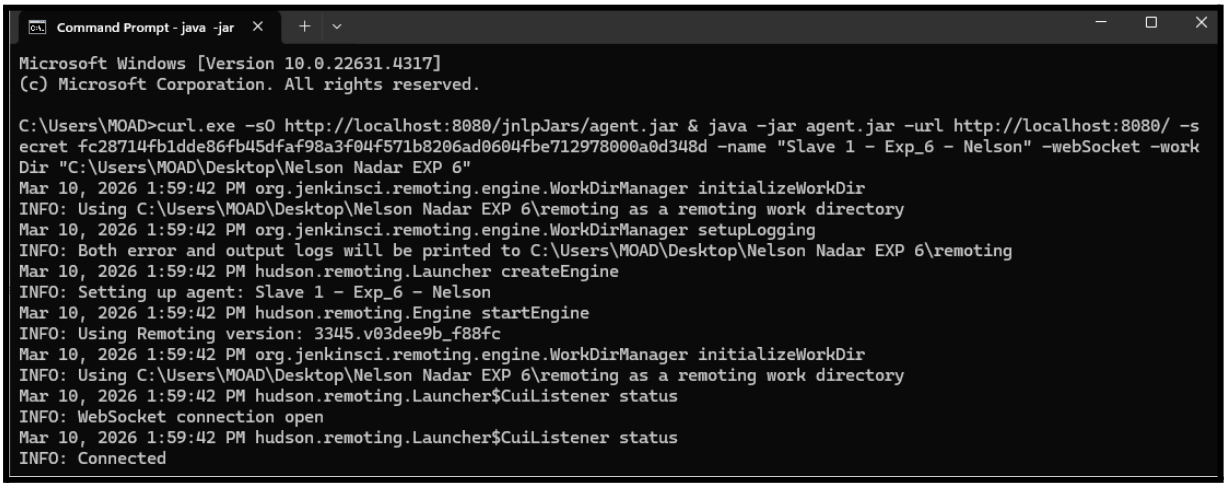
Give slave name, Number of executors =2, remote root directory = C:\Vaishali DevOps 2026, label = S1

Check slave is created but it is offline

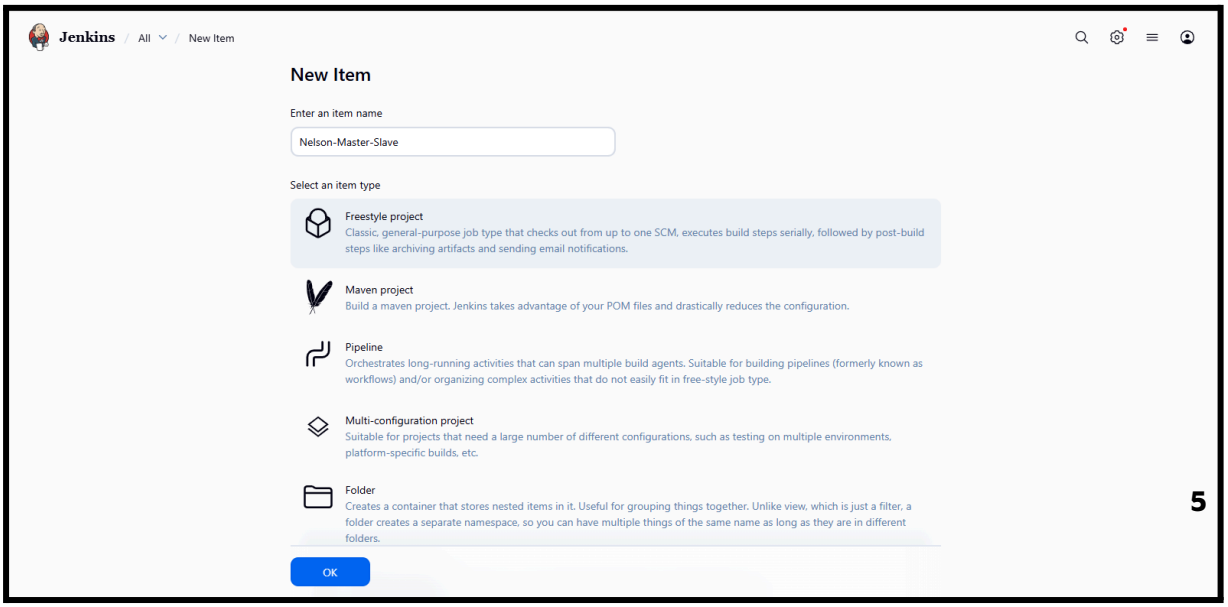
Click on Slave 1, there is a code to run on windows command prompt, copy it and paste it on cmd.



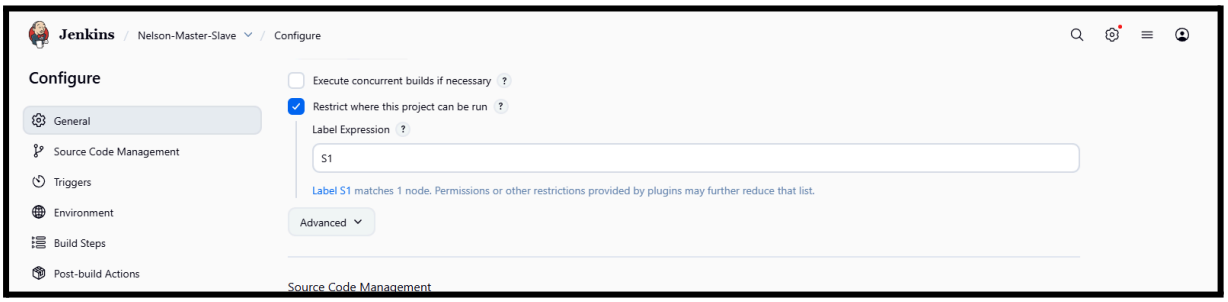
Slave 1 is connected



Check Jenkins ->manage Jenkins->nodes , slave 1 is connected now go to Jenkins New item-> Give name of project as Master-Slave , Freestyle project ->ok



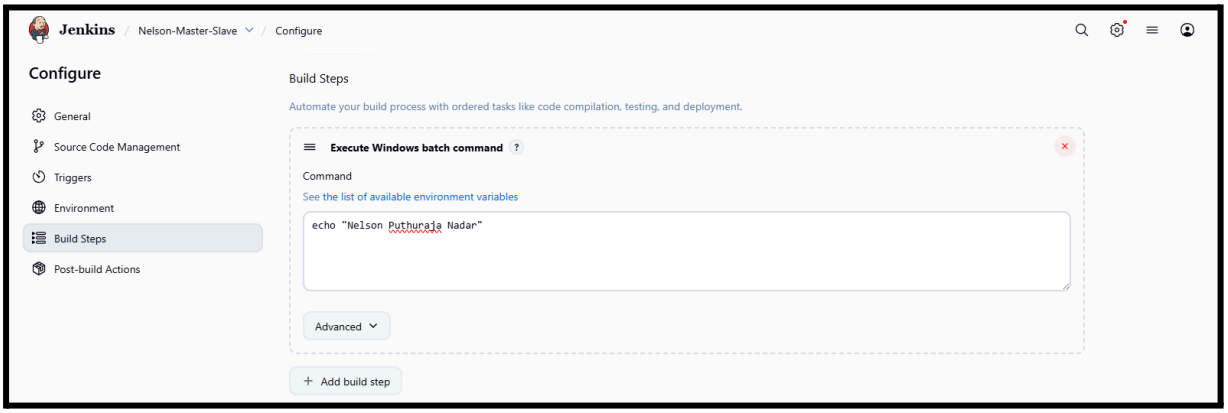
Enable Restrict Project can be run and give label Name as S1 (short form of slave node while creating node)



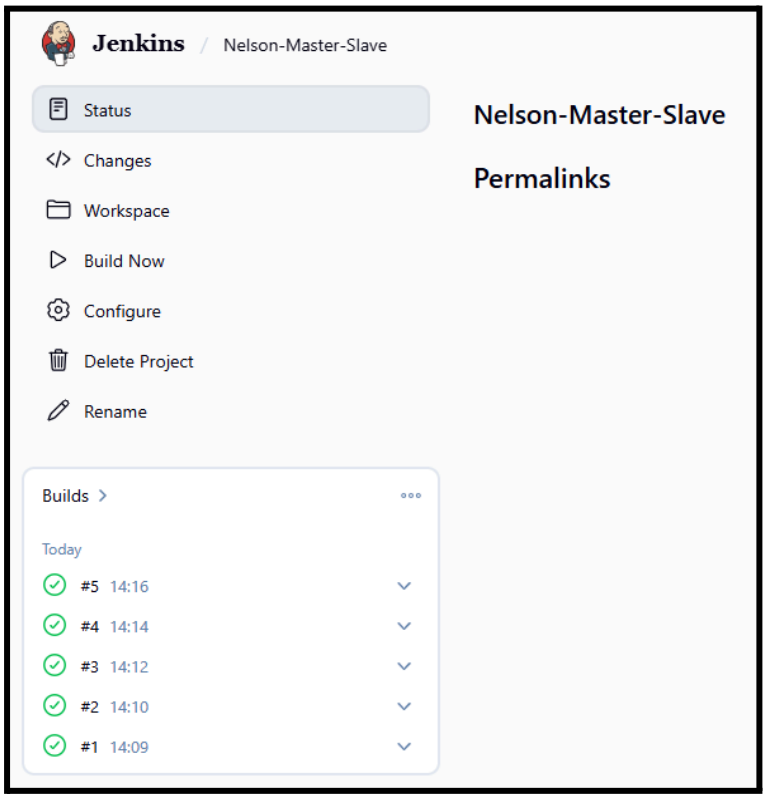
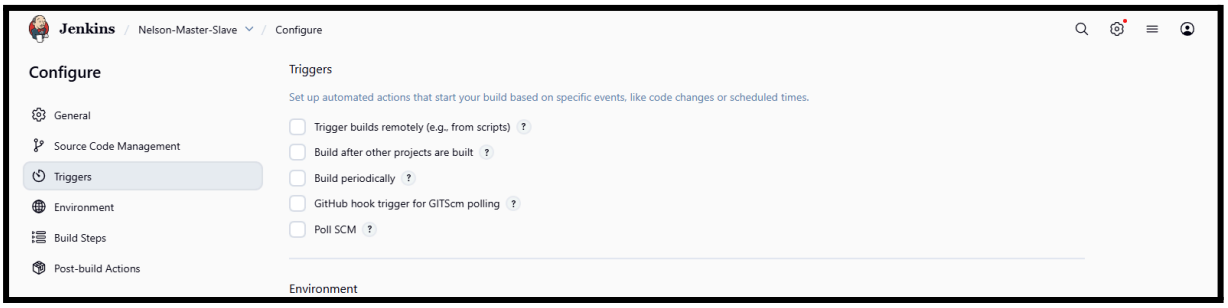
Scroll and go to Triggers -> click on Build Periodically

Google Jenkins Pipeline Syntax and search for Jenkins cron syntax
There are values as Minutes, Hours, Date Of Month , Month, Date of week
We will set the value as Hour /2 means the project will build after every 2 minutes.
Remaining values we will keep as null ie *.
Give Schedule Value as H/2 * * * *

Give Build Steps as Execute Windows batch command
Give Command as Echo "Hello Vaishali"
Apply and save and build now



Check Jenkins master is building the project after 2 mins with the help of slave or not



Go to configure and triggers -> without fail unclick the build periodically option of the project, otherwise Jenkins master will get triggered after every 2 min.